چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۳

CI/CD و DevOps — بخش اول: پایپ‌لاین، کش، گیت‌های کیفیت و تست روی دستگاه



اسماعیل مرادی

@esmaeeil-moradi700

چرا CI/CD در ۲۰۲۶ مهم است؟

از یک مهندس سینیور اندروید انتظار می‌رود مالک مسیر از commit تا انتشار باشد، نه فقط کد. مصاحبه‌گر می‌سنجد که پایپ‌لاین شما **سریع، قابل‌اعتماد و امن** است یا نه:

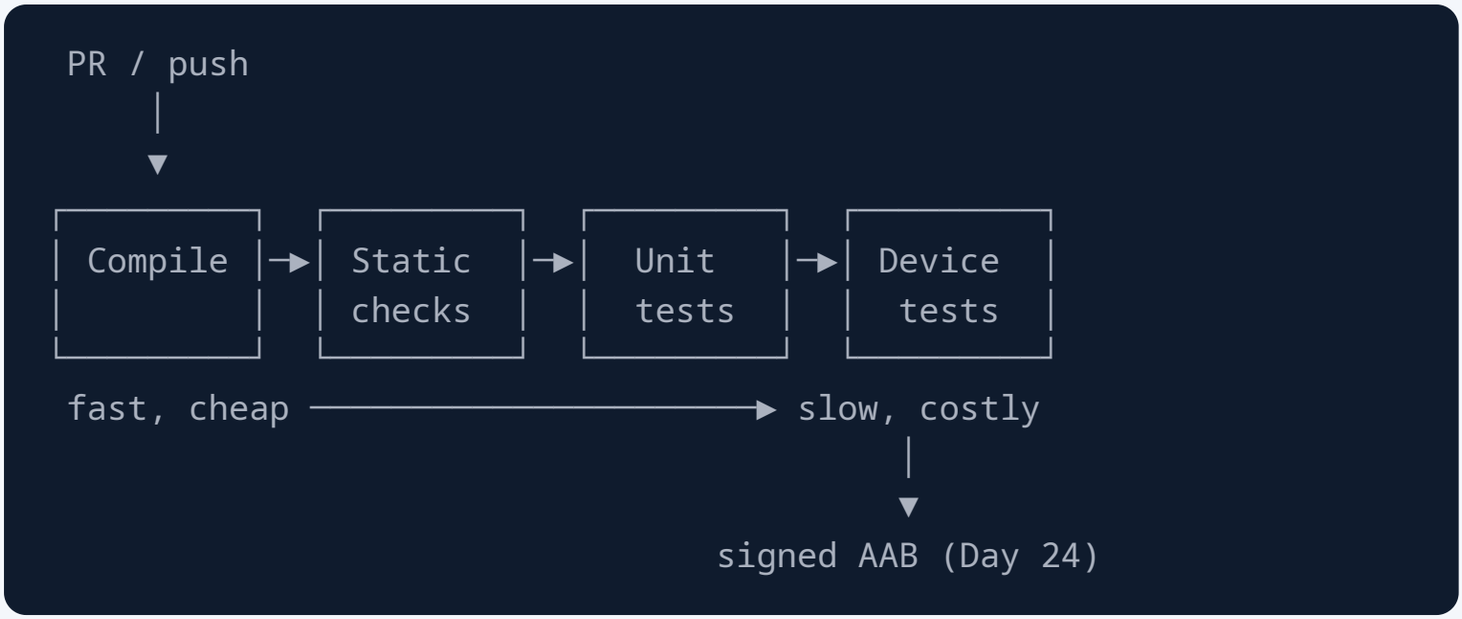
- هر merge به `main` باید قابل انتشار باشد — تصمیم با پایپ‌لاین است، نه با یک آدم
- buildهای Gradle کندند: **کش** و **شاردینگ** CI را قابل‌استفاده می‌کنند
- تحلیل ایستا به‌عنوان **گیت** (Lint، detekt، ktlint)، نه گزارشی که کسی نمی‌خواند
- تست روی دستگاه واقعی با **Gradle Managed Devices** یا **Firebase Test Lab**
- خود پایپ‌لاین یک **سطح حمله** است (امنیت زنجیرهٔ تأمین)

سطح	سؤال‌ها
جونیور	CI در برابر CD، اولین workflow، گیت PR
مید	کش Gradle، تحلیل ایستا، تست دستگاه در CI، شاردینگ
سینیور	پایپ‌لاین برای هر trigger، تست‌های flaky، کدام گیت merge را متوقف کند
آرکیتکت	پایپ‌لاین قابل‌حمل و runnerها، امنیت زنجیرهٔ تأمین، درمان CI کند



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: اول چک‌های ارزان



- **زود شکست بخور:** ارزان‌ترین و محتمل‌ترین خطاها اول اجرا می‌شوند
- **موازی کن:** jobهای مستقل (lint و unit test) کنار هم اجرا می‌شوند
- **همه‌جا همان taskها:** CI همان taskهای Gradle را صدا می‌زند که توسعه‌دهنده روی سیستم خودش اجرا می‌کند

ایدهٔ واحد پشت هر پایپ‌لاین

پایپ‌لاین یک **سیستم بازخورد** است. هر مرحله را با دو سؤال بسنجید: چقدر سریع می‌گوید چیزی خراب است، و وقتی قرمز است می‌شود به آن اعتماد کرد؟



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

CI در برابر CD، اولین workflow، گیت PR



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱. CI، Continuous Delivery و Continuous Deployment

اصطلاح	یعنی	در اندروید
Continuous Integration	همه تغییرات کوچک را مرتب در `main` ادغام می‌کنند؛ هر تغییر خودکار build و تست می‌شود	چک‌های PR، به‌علاوهٔ build هر commit روی `main`
Continuous Delivery	هر build سبز `main` **می‌تواند** منتشر شود؛ ساخت artifact امضاشده خودکار است و انتشار یک تصمیم است	AAB خودکار روی track داخلی آپلود می‌شود
Continuous Deployment	هر build سبز بدون دخالت انسان به کاربر می‌رسد	در موبایل کمیاب است: بازبینی فروشگاه، به‌روزرسانی باینری توسط کاربر، نبود rollback

تلهٔ مصاحبه

«*CI یعنی سروری که کد همه را خودکار در main ادغام می‌کند.*» (جملهٔ نسخهٔ پارسال) — **نه.** توسعه‌دهنده‌ها ادغام می‌کنند؛ CI هر ادغام را **وارسی** می‌کند. CI یک عادت کاری است — mergeهای کوچک و مکرر. تیمی که Jenkins دارد و branchهای سه‌هفته‌ای، یک build server دارد، نه CI.

تلهٔ مصاحبه

«*ما روی Google Play continuous deployment داریم.*» — معمولاً continuous **delivery** به یک track تستی است و بعد یک ارتقای آگاهانه. در موبایل نمی‌شود انتشار را پس گرفت (روز ۲۴).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. اولین workflow در GitHub Actions

```yaml
name: ci
on:
  pull_request:
  push:
    branches: [main]
jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew assembleDebug testDebugUnitTest lintDebug
```

- `setup-gradle` پوشهٔ Gradle User Home را کش می‌کند و گزارش build می‌دهد — بدون کلید کش دست‌ساز
- نسخه‌های فعلی AGP به JDK 17 نیاز دارند؛ همیشه با **wrapper** بسازید (`./gradlew`)
- فایل در `.github/workflows/ci.yml` قرار می‌گیرد؛ تگ‌های بالا نسخهٔ اصلی‌اند — نسخهٔ جدیدتر را چک کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«`./gradlew test` *یعنی اجرای unit testها.*» — این دستور تست‌ها را برای **همهٔ variantها** اجرا می‌کند: debug، release و هر flavor. روی PR، `testDebugUnitTest` همان سیگنال را در کسری از زمان می‌دهد.

تلهٔ مصاحبه

در نسخهٔ پارسال: «`connectedCheck` *همهٔ تست‌های instrumented و چک‌های lint را اجرا می‌کند.*» — **اشتباه است.** `connectedCheck` فقط تست‌های instrumented را روی دستگاه‌های وصل‌شده اجرا می‌کند. برای Lint یک task جدا به نام `lintDebug` وجود دارد.

سؤال تکمیلی

«چرا `timeout-minutes` *را تنظیم کنیم؟*» — timeout پیش‌فرض هر job در GitHub Actions شش ساعت است. یک امولاتور گیرکرده یا یک تست قفل‌شده، قبل از اینکه کسی متوجه شود ۶ ساعت از وقت runner را می‌سوزاند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. روی هر pull request چه اجرا می‌شود؟

گیت status check — **PR**های الزامی در branch protection یا rulesetها. هیچ چیز قرمز merge نمی‌شود.

- **کامپایل** variantهایی که منتشر می‌کنید (روی PR دست‌کم debug)
- **unit test** برای کد تغییرکرده
- **Lint**، **detekt** و **ktlint** — مشکلات سبک و الگوهای باگ همان دمِ در متوقف می‌شوند
- **تست‌های معماری** (Konsist، روز ۳) — ارزان و قطعی
- روی `main` (یا شبانه): یک **build نسخهٔ release با R8**، چون باگ‌های shrink و obfuscation فقط آنجا دیده می‌شوند

تلهٔ مصاحبه

«*همه‌چیز را روی هر PR اجرا کن.*» — ماتریس کامل امولاتور روی هر push بازخورد را کند می‌کند و توسعه‌دهنده‌ها تغییرات را در PRهای بزرگ جمع می‌کنند. این دقیقاً برعکس CI است. چک‌های PR را در حد چند دقیقه نگه دارید؛ مجموعه‌های گران را به merge یا اجرای شبانه ببرید.

زمان بازخورد یک فیچر است

اگر CI کند باشد، آدم‌ها دیگر منتظرش نمی‌مانند. یک build قرمز سریع و قابل‌اعتماد از یک build کامل که کسی به آن اعتماد ندارد باارزش‌تر است.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

کش Gradle، تحلیل ایستا، تست دستگاه در CI، شاردینگ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. کش Gradle در CI

لایه	چه چیزی را صرفه‌جویی می‌کند	چطور
Gradle User Home	وابستگی‌های دانلودشده و توزیع wrapper	در GitHub با `setup-gradle`؛ در GitLab با انتقال `GRADLE_USER_HOME` به داخل پروژه و بخش `cache:`
Build cache	خروجی taskها (کامپایل، dex، تست) که بین buildها و ماشین‌ها استفادهٔ مجدد می‌شود	تنظیم `org.gradle.caching=true` و یک remote cache یا Develocity
Configuration cache	گراف taskهای پیکربندی‌شده — مرحلهٔ configuration حذف می‌شود	تنظیم `org.gradle.configuration-cache=true`؛ `setup-gradle` فقط با کلید رمزنگاری ذخیره‌اش می‌کند

- به‌طور پیش‌فرض `setup-gradle` کش را فقط از branch پیش‌فرض **می‌نویسد**؛ buildهای PR فقط می‌خوانند
- کش محلی و ریموت فقط برای taskهای **قابل‌کش و بازتولیدپذیر** خروجی را دوباره استفاده می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. remote build cache — و تله‌هایش

```kotlin
// settings.gradle.kts
val isCi = providers.environmentVariable("CI").isPresent

buildCache {
    remote<HttpBuildCache> {
        url = uri("https://gradle-cache.example.com/cache/")
        isPush = isCi
        credentials {
            username = providers
                .environmentVariable("CACHE_USER").orNull
            password = providers
                .environmentVariable("CACHE_PASSWORD").orNull
        }
    }
}
```

تلهٔ مصاحبه

اجازهٔ push به remote cache از لپ‌تاپ توسعه‌دهنده‌ها. یک لپ‌تاپ با تنظیمات اشتباه می‌تواند کش را برای همه مسموم کند. **فقط CI push می‌کند؛ لپ‌تاپ‌ها فقط pull.**

تلهٔ مصاحبه

یک `buildConfigField` با زمان build یا hash گیت در هر build دیباگ، ورودی همهٔ buildها را عوض می‌کند — نرخ hit کش به صفر می‌رسد. این مقدارها را از build دیباگ بیرون نگه دارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. تحلیل ایستا به‌عنوان گیت

ابزار	چه چیزی را می‌گیرد	task در Gradle
Android Lint	باگ‌های مخصوص اندروید: `NewApi`، permission جاافتاده، resource بلااستفاده، دسترس‌پذیری، امنیت	`lintDebug`
detekt	code smellهای Kotlin، پیچیدگی، باگ‌های محتمل؛ قاعدهٔ سفارشی	`detekt`
ktlint	قالب‌بندی و سبک رسمی Kotlin	`ktlintCheck` در پلاگین ktlint-gradle، یا `spotlessCheck`
قواعد Compose	استفادهٔ اشتباه از API کامپوز: پارامتر modifier، hoisting، نام‌گذاری	قواعد compose-rules برای ktlint و detekt، یا compose-lints از Slack

تلهٔ مصاحبه

نسخهٔ پارسال `./gradlew detektCheck` را اجرا می‌کرد — **چنین taskی وجود ندارد.** task پلاگین detekt همان `detekt` است (به‌علاوهٔ نسخه‌های type-resolution مثل `detektMain`).

بگذارید FORMATTER برنده شود

قبل از commit دستور `ktlintFormat` یا Spotless را اجرا کنید تا code review دربارهٔ طراحی باشد، نه فاصله‌ها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. تنظیم گیت‌ها

```kotlin
// app/build.gradle.kts
android {
    lint {
        abortOnError = true
        checkDependencies = true
        baseline = file("lint-baseline.xml")
        sarifReport = true
    }
}

detekt {
    buildUponDefaultConfig = true
    config.setFrom("$rootDir/config/detekt.yml")
    baseline = file("detekt-baseline.xml")
}
```

- **baseline** مشکلات امروز را منجمد می‌کند؛ فقط مشکلات **جدید** build را fail می‌کنند — راه اضافه کردن گیت به یک کدبیس قدیمی
- گزارش‌های **SARIF** را می‌شود در GitHub code scanning آپلود کرد تا یافته‌ها روی diff همان PR نمایش داده شوند

تلهٔ مصاحبه

بازسازی baseline «برای اینکه CI سبز شود». حالا مشکلات جدید هم پنهان شده‌اند. baseline باید **فقط کوچک شود** — اندازه‌اش را مثل بدهی فنی پیگیری کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. تست instrumented در CI: سه گزینه

گزینه	اجرا روی	مناسب برای	مراقب باشید
امولاتور روی runner	runner لینوکسی با KVM فعال	مجموعه‌های کوچک، کنترل کامل	زمان boot، پردازندهٔ runner، flaky بودن
Gradle Managed Devices	امولاتورهایی که AGP می‌سازد و مدیریت می‌کند	تعریف در Gradle؛ همان تنظیمات روی سیستم محلی و CI	باز هم امولاتور است؛ KVM لازم دارد
Firebase Test Lab	دستگاه‌های فیزیکی و مجازی گوگل	سخت‌افزار واقعی، سطوح API متعدد، شاردینگ	هزینه و سهمیه، بازخورد کندتر

- برای امولاتور روی runner معمولاً از اکشن `reactivecircus/android-emulator-runner` استفاده می‌شود

تلهٔ مصاحبه

«*همهٔ تست‌های دستگاه را روی همهٔ سطوح API برای هر PR اجرا کن.*» — برای PR یک دستگاه نماینده انتخاب کنید. ماتریس کامل (سطوح API، اندازهٔ دستگاه، دستگاه واقعی) را روی `main` یا شبانه اجرا کنید.

سؤال تکمیلی

«*چرا KVM مهم است؟*» — imageهای x86 امولاتور به شتاب‌دهی سخت‌افزاری نیاز دارند؛ بدون آن امولاتور یا بالا نمی‌آید یا به‌شکل غیرقابل‌استفاده‌ای کند است. روی runnerهای لینوکسی GitHub، قبل از بالا آوردن امولاتور با یک مرحلهٔ udev rule دسترسی به KVM را باز می‌کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. Gradle Managed Devices

```
android {
    testOptions {
        animationsDisabled = true
        execution = "ANDROIDX_TEST_ORCHESTRATOR"
        managedDevices {
            localDevices {
                create("pixel6Api34") {
                    device = "Pixel 6"
                    apiLevel = 34
                    systemImageSource = "aosp-atd"
                }
            }
        }
    }
}
```

```
./gradlew pixel6Api34DebugAndroidTest
```

- imageهای **ATD**، یعنی Automated Test Device، نسخه‌های سبک‌شدهٔ سیستم برای تست بدون نمایشگرند
- **Orchestrator** هر تست را در instrumentation جداگانه اجرا می‌کند — کرش یک تست کل مجموعه را از کار نمی‌اندازد (وابستگی `androidx.test:orchestrator` را با `androidTestUtil` اضافه کنید)

تلهٔ مصاحبه

imageهای ATD بعضی اپ‌ها و سرویس‌های سیستمی را حذف کرده‌اند. تستی که به آن‌ها نیاز دارد فقط روی ATD fail می‌شود — آن مجموعه را به image معمولی `aosp` یا `google` ببرید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. شاردینگ تست: کوتاه کردن زمان واقعی

- شاردینگ زمان **واقعی (wall-clock)** را کم می‌کند، نه زمان ماشین را: هر شارد هزینهٔ checkout، راه‌اندازی Gradle و boot امولاتور را جداگانه می‌پردازد
- **unit test:** اول forkهای موازی داخل یک job؛ تقسیم ماژول‌ها بین jobها فقط در ریپوهای بزرگ
- **تست دستگاه:** خود AndroidJUnitRunner با `numShards` و `shardIndex` شارد می‌کند

```
// build-logic convention plugin
tasks.withType<Test>().configureEach {
    maxParallelForks =
        (Runtime.getRuntime().availableProcessors() / 2)
            .coerceAtLeast(1)
}
```

تلهٔ مصاحبه

«شاردهای یکنواخت متوازن‌اند.» — شاردینگ یکنواخت بر اساس **تعداد تست** تقسیم می‌کند، نه مدت‌زمان. یک کلاس کند، یک شارد را گلوگاه می‌کند. شاردینگ مبتنی بر زمان (مثلاً Flank) بر اساس زمان اجراهای قبلی متوازن می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. شاردینگ تست دستگاه بین چند ماشین

```yaml
  device-tests:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        shard: [0, 1, 2, 3]
    steps:
      # checkout, JDK, setup-gradle, enable KVM …
      - run: ./scripts/device-tests.sh ${{ matrix.shard }} 4
```

```
# scripts/device-tests.sh <shardIndex> <numShards>
ARGS=-Pandroid.testInstrumentationRunnerArguments
./gradlew pixel6Api34DebugAndroidTest \
  "$ARGS.shardIndex=$1" "$ARGS.numShards=$2"
```

- به‌جایش یک ماشین بزرگ: GMD می‌تواند با `android.experimental.androidTest.numManagedDeviceShards` بین چند امولاتور شارد کند
- در Firebase Test Lab از فلگ `--num-uniform-shards` در دستور `gcloud firebase test android run` استفاده کنید
- با `fail-fast: false` همهٔ شاردها تا آخر اجرا می‌شوند و همهٔ خطاها را یک‌جا می‌بینید



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

پایپ‌لاین برای هر trigger، تست‌های flaky، کدام گیت merge را متوقف کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. طراحی پایپ‌لاین برای هر trigger

رویداد	چه اجرا می‌شود	هدف
pull request	کامپایل، unit test، Lint و detekt و ktlint — فقط ماژول‌های متأثر	بازخورد در چند دقیقه
صف ادغام — رویداد `merge_group`	همان چک‌ها روی PR **ترکیب‌شده با آخرین `main`**	دو PR سبز با هم `main` را نمی‌شکنند
push به `main`	کل unit testها، build نسخهٔ release با R8، تست دود روی دستگاه، آپلود به internal testing	`main` همیشه قابل انتشار است
شبانه — رویداد `schedule`	ماتریس دستگاه روی Firebase Test Lab، تست screenshot، بنچمارک	پوشش گران‌قیمت
تگ یا release branch	AAB امضاشده، release notes، آپلود به Play	روز ۲۴

سؤال تکمیلی

«*چرا صف ادغام (merge queue)؟*» — دو PR ممکن است هر کدام روی یک `main` قدیمی پاس شوند و باز هم از نظر معنایی تداخل داشته باشند (یکی تابعی را rename می‌کند که دیگری صدا می‌زند). صف، آن‌ها را به ترتیب ادغام تست می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. triggerها و concurrency در YAML

```yaml
on:
  pull_request:
  merge_group:
  push:
    branches: [main]
  schedule:
    - cron: "0 2 * * *"   # nightly, UTC
  workflow_dispatch:

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}
```

- push جدید روی یک PR، اجرای قدیمی را **لغو** می‌کند؛ اجراهای روی `main` هیچ‌وقت لغو نمی‌شوند — هر commit یک کاندیدای انتشار است

تلهٔ مصاحبه

فیلتر `paths:` روی workflowی که یک check **الزامی** تولید می‌کند. وقتی workflow رد می‌شود، check در حالت *pending* می‌ماند و PR قابل merge نیست. فیلتر را داخل job بگذارید، یا یک job نهایی «گیت» اضافه کنید که همیشه نتیجه گزارش کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تست‌های flaky: قرمز را معنادار نگه دارید

تست flaky روی **همان کد** گاهی پاس و گاهی fail می‌شود. هزینهٔ واقعی‌اش: آدم‌ها آن‌قدر اجرا را تکرار می‌کنند تا سبز شود، و بعد خطاهای واقعی را هم نادیده می‌گیرند.

مرحله	چطور
شناسایی	نتیجهٔ هر تست را در طول اجراها دنبال کنید (تاریخچهٔ JUnit XML، Develocity، نتایج Test Lab)
مهار	تعداد محدودی retry — و گزارش «پاس بعد از retry» به‌عنوان flaky
قرنطینه	انتقال به مجموعه‌ای غیرمسدودکننده، با مالک و مهلت
رفع	state مشترک، زمان یا شبکهٔ واقعی، انیمیشن، نبود idling resource، ترتیب تست‌ها

```
// plugins { id("org.gradle.test-retry") }
tasks.withType<Test>().configureEach {
    retry {
        maxRetries.set(2)
        failOnPassedAfterRetry.set(false)
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. چیزهایی که مصاحبه‌گر چک می‌کند

- تست دستگاه: Orchestrator با `clearPackageData` state را بین تست‌ها جدا می‌کند
- در Firebase Test Lab، فلگ `--num-flaky-test-attempts` تست‌های failشده را دوباره اجرا می‌کند
- **نرخ flaky** را به‌عنوان یک شاخص سلامت CI دنبال کنید (سؤال ۱۳)

تلهٔ مصاحبه

«*فقط retry اضافه کن.*» — retry بی‌صدا باگ‌های واقعی را پنهان می‌کند: race condition در تست اغلب همان race condition در production است. retryها باید دیده، شمرده و صاحب‌دار شوند.

تلهٔ مصاحبه

`@Ignore` بدون مالک یا تیکت. تست ignoreشده یک تست حذف‌شده است که هنوز هزینهٔ کامپایل دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. کدام گیت‌ها باید merge را متوقف کنند؟

گیت	PR را متوقف کند؟	چرا
کامپایل، unit test، خطاهای Lint، detekt، ktlint	**بله**	ارزان و قطعی
تست‌های معماری (Konsist، روز ۳)	**بله**	از مرز ماژول‌ها و لایه‌ها محافظت می‌کند
تست screenshot با Roborazzi یا Paparazzi (روز ۱۰)	**بله**، با بازبینی diff تصاویر	روی JVM اجرا می‌شود — سریع
پوشش تست (coverage)	به‌عنوان **سیگنال** روی خطوط تغییرکرده	تلهٔ صفحهٔ بعد را ببینید
حجم اپ	هشدار وقتی رشد از بودجه بیشتر شود	build نسخهٔ release لازم دارد
بنچمارک	شبانه با تشخیص افت (روز ۲۸)	روی runnerهای مشترک پرنویز است
به‌روزرسانی وابستگی‌ها	PRهای Renovate یا Dependabot از همان گیت عبور می‌کنند	ارتقاهای کوچک و مکرر



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. تلهٔ coverage

تلهٔ مصاحبه

پارسال: «*اگر coverage زیر ۷۵ درصد رفت build را fail کن — این تضمین می‌کند کد جدید همیشه تست دارد.*» — coverage خطوط **اجراشده** را می‌سنجد، نه رفتار **وارسی‌شده**. تستی بدون هیچ assertion می‌تواند به ۱۰۰ درصد برسد. آستانهٔ سراسری به تست‌های بی‌ارزش جایزه می‌دهد و حذف کد مرده را جریمه می‌کند.

- راه بهتر: **diff coverage** روی خطوط تغییرکرده به شکل کامنت روی PR، به‌علاوهٔ قضاوت بازبین
- ابزارها: **Kover** با task `koverVerify`، یا JaCoCo داخلی AGP — با `enableUnitTestCoverage = true` روی یک build type، task `createDebugUnitTestCoverageReport` ساخته می‌شود
- task `jacocoTestReport` از پلاگین `jacoco` خود Gradle برای ماژول‌های JVM می‌آید — AGP آن را برای ماژول‌های اندروید نمی‌سازد

سؤال تکمیلی

«*پس coverage بی‌فایده است؟*» — نه. coverage **پایین** در کد حیاتی یک هشدار واقعی است. coverage **بالا** به‌تنهایی چیزی را ثابت نمی‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

پایپ‌لاین قابل‌حمل و runnerها، امنیت زنجیرهٔ تأمین، درمان CI کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. منطق در Gradle، YAML لاغر

همان taskهای Gradle، یک CI دیگر:

```yaml
# .gitlab-ci.yml
variables:
  GRADLE_USER_HOME: "$CI_PROJECT_DIR/.gradle"
cache:
  key:
    files: [gradle/wrapper/gradle-wrapper.properties]
  paths: [.gradle/caches/, .gradle/wrapper/]
check:
  image: $ANDROID_BUILD_IMAGE   # JDK 17 + Android SDK
  script:
    - ./gradlew assembleDebug testDebugUnitTest lintDebug
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

- GitLab فقط مسیرهای **داخل** پوشهٔ پروژه را کش می‌کند — دلیل جابه‌جایی `GRADLE_USER_HOME` همین است
- YAML فقط ماشین را آماده می‌کند و Gradle را صدا می‌زند. منطق build در convention pluginها زندگی می‌کند (روز ۲۱)، پس توسعه‌دهنده‌ها می‌توانند آن را روی سیستم خودشان اجرا کنند و شما می‌توانید سرویس CI را عوض کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. انتخاب پلتفرم و runner

انتخاب	نقطهٔ قوت	هزینه و ریسک
GitHub Actions میزبانی‌شده	بدون نگهداری، marketplace، صف ادغام	هزینهٔ دقیقه‌ای؛ دقیقه‌های macOS برای iOS و KMP گران‌ترند
GitLab CI	کد، MR، CI و registry در یک‌جا؛ runnerهای self-managed پخته	کش باید داخل پوشهٔ پروژه باشد
runnerهای self-hosted یا autoscale	ماشین بزرگ، کش گرم، شبکهٔ خصوصی	patch و ایزوله‌سازی با خودتان است
سرویس‌های CI موبایل مثل Bitrise و Codemagic	ابزارهای اندروید و iOS از قبل نصب‌اند	وابستگی به stepهای اختصاصی فروشنده

تلهٔ مصاحبه

پارسال: «*یک remote cache توزیع‌شدهٔ جهانی همه‌چیز را عوض می‌کند.*» — فقط اگر taskها قابل‌کش باشند و کش **نزدیک runnerها** باشد. کشی در قارهٔ دیگر می‌تواند از build دوباره کندتر باشد. اول نرخ hit و زمان دانلود را اندازه بگیرید.

تلهٔ مصاحبه

runner از نوع self-hosted روی ریپوی **عمومی**: هر PR از یک fork می‌تواند روی ماشین شما کد اجرا کند و چیزی جا بگذارد. از runnerهای یک‌بارمصرف استفاده کنید، و هرگز برای PRهای غیرقابل‌اعتماد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. پایپ‌لاین یک سطح حمله است

```yaml
permissions:
  contents: read      # least privilege by default

on:
  pull_request:       # fork PRs: no secrets, read-only token
# pull_request_target runs with secrets and a write token:
# never check out and build PR code under it
```

```
# Record checksums of every dependency and plugin;
# builds fail on mismatch. Review the file in PRs.
./gradlew --write-verification-metadata sha256 help
```

تلهٔ مصاحبه

«*ریپوی ما خصوصی است، پس امنیت CI مهم نیست.*» — در مارس ۲۰۲۵ اکشن پرطرفدار `tj-actions/changed-files` آلوده شد: هر workflowی که از تگ آن استفاده می‌کرد کدی اجرا کرد که secretها را در لاگ build چاپ می‌کرد. راه‌حل: **اکشن‌های شخص ثالث را به commit SHA کامل pin کنید** و pinها را با Dependabot به‌روز نگه دارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. جعبه‌ابزار زنجیرهٔ تأمین

ریسک	کنترل
اکشن شخص ثالث آلوده	pin به commit SHA؛ به‌روزرسانی خودکار pinها
توکن `GITHUB_TOKEN` با دسترسی زیاد	بخش `permissions:` پیش‌فرض فقط‌خواندنی، دسترسی نوشتن فقط برای job لازم
رسیدن secret به کد غیرقابل‌اعتماد	رویداد `pull_request` برای forkها؛ secretهای انتشار در environmentهای محافظت‌شده (روز ۲۴)
فایل jar آلوده در Gradle wrapper	اعتبارسنجی wrapper با `gradle/actions/wrapper-validation`
وابستگی دست‌کاری‌شده	dependency verification در Gradle؛ فیلتر محتوای repositoryها
runner ماندگار self-hosted	runnerهای یک‌بارمصرف، هر کدام یک job

نظر من

با پایپ‌لاین مثل کد production رفتار کنید: بازبینی‌شده، با حداقل دسترسی، pinشده و قابل ممیزی. job انتشار کلید امضا و دسترسی Play را در اختیار دارد — باارزش‌ترین هدف در کل زنجیرهٔ اندرویدی شرکت است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. «CI ما ۴۰ دقیقه طول می‌کشد. درستش کن.»

۱. **اول اندازه بگیرید.** با build scan، یعنی `--scan` یا Develocity، ببینید زمان کجا می‌رود — صف، راه‌اندازی، configuration، کامپایل، تست، boot امولاتور؟

۲. **کار نکنید.** configuration cache، remote build cache، `testDebugUnitTest` به‌جای `test`، و فقط **ماژول‌های متأثر** — مثلاً با ابزار Affected Module Detector از Dropbox.

۳. **موازی کنید.** jobهای مستقل کنار هم؛ شاردینگ تست‌های دستگاه.

۴. **کار را جابه‌جا کنید.** مجموعه‌های گران به صف ادغام یا اجرای شبانه.

۵. **زمان بخرید.** runner بزرگ‌تر اغلب از وقت مهندس‌هایی که منتظرش هستند ارزان‌تر است.

۶. **ساختار را درست کنید.** یک ماژول یکپارچهٔ `:app` جلوی همهٔ کش‌ها را می‌گیرد — ماژولار کردن موضوع روز ۲۵ است.

تلهٔ مصاحبه

«*تست فقط ماژول‌های متأثر همیشه امن است.*» — فقط اگر گراف وابستگی درست باشد. تغییر در build logic، version catalog یا resourceهای مشترک باید همه‌چیز را اجرا کند — و `main` همچنان کل مجموعه را اجرا می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. شاخص‌های یک CI سالم

شاخص	چه می‌گوید
زمان بازخورد PR (میانه و صدک ۹۰)	چیزی که توسعه‌دهنده واقعاً حس می‌کند
زمان صف	مشکل ظرفیت runner، نه مشکل build
نرخ hit کش build	اینکه کش واقعاً کار می‌کند یا نه
نرخ خطای flaky	اینکه می‌شود به build قرمز اعتماد کرد یا نه
مدت قرمز بودن `main`	چقدر «همیشه قابل انتشار» دروغ است
هزینهٔ هر PR	ورودی تصمیم‌های بودجه

جواب سینیور در یک جمله

«CI محصولی برای توسعه‌دهنده‌هاست: بازخورد سریع روی هر PR، build قرمزی که می‌شود به آن اعتماد کرد، چک‌های گران جایی که کمترین هزینه را دارند — و پایپ‌لاینی به همان اندازه قفل‌شده که انتشاری که می‌سازد.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۳ — خلاصهٔ یک‌صفحه‌ای

- CI یعنی mergeهای کوچک و مکرر با وارسی خودکار. در موبایل معمولاً continuous **delivery** داریم، نه deployment.
- اول چک‌های ارزان؛ CI همان taskهای Gradle توسعه‌دهنده را صدا می‌زند.
- به‌جای `test` از `testDebugUnitTest` استفاده کنید؛ `connectedCheck` فقط تست دستگاه است — Lint جداست.
- `timeout-minutes` را تنظیم کنید؛ اجراهای قدیمی PR را لغو کنید، اجراهای `main` را هرگز.
- پوشهٔ Gradle User Home را با `setup-gradle` کش کنید؛ در remote build cache فقط CI push می‌کند.
- Lint، detekt و ktlint به‌عنوان check الزامی — task درست `detekt` است، نه `detektCheck`؛ baseline فقط کوچک می‌شود.
- تست دستگاه: یک دستگاه روی PR، ماتریس در اجرای شبانه — با GMD یا Firebase Test Lab؛ شاردینگ با `numShards` و `shardIndex`.
- تست flaky را با retry قابل‌مشاهده مهار و با مالک قرنطینه کنید؛ ریشه را رفع کنید.
- coverage سیگنال است، نه گیت. صف ادغام از `main` محافظت می‌کند.
- اکشن‌ها pinشده به SHA، توکن با حداقل دسترسی، بدون secret برای PRهای fork، و dependency verification.

روز ۲۴ در راه است: CI/CD بخش دوم — امضا، انتشار و rollout. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700